logo
URCA

Université de Reims Champagne-Ardenne
UFR Sciences économiques, sociales et de gestion
Master Analyse et politique économique
Parcours Statistique pour l'évaluation et la prévision
Master SEP

KataML

Rapport des modifications

Adaptation à macOS, chargement des données et des bases de données, import de scripts Python, performances

Réalisé par Votre nom, étudiant en master SEP
Encadrant Nom du professeur
Formation Master Analyse et politique économique, parcours SEP
Année universitaire 2026–2027
Date 25 octobre 2026

Application KataML, atelier visuel de machine learning (scikit-learn)
Version de départ 0.5.8 (16 octobre 2026), licence MIT
Dépôt https://gitlab.com/fredbl/kataml

Ce document décrit les fonctionnalités ajoutées à KataML dans le cadre du master Statistique pour l'évaluation et la prévision, leur fonctionnement interne, les fichiers modifiés et les tests écrits. Il peut accompagner une demande de fusion (*merge request*) sur le dépôt GitLab du projet.

Table des matières

Résumé **2**

1 Contexte et objectifs **2**

2 Fonctionnement sur macOS **3**

3 Chargement des données **3**

3.1 Lecteur Parquet / Excel 3

3.2 Lecteur SQL 3

3.3 Import des fichiers 4

4 Import d'un script Python en schéma **4**

4.1 Principe 4

4.2 Valeurs suivies 4

4.3 Éléments reconnus 5

4.4 Points notables 5

4.5 Exemple : le projet de retards de vols 6

4.6 Limites 6

5 Espace « Données et scripts » **6**

6 Performances **7**

6.1 Calcul sur tous les cœurs 7

6.2 Prédictions hors pli sans double entraînement 7

6.3 Mesures 8

7 Encodeurs et colonnes texte **8**

8 Traduction et documentation **8**

9 Tests **9**

10 Fichiers modifiés **10**

11 Perspectives **10**

A Commandes utiles **10**

Résumé

KataML est un atelier visuel de machine learning : on y construit une analyse en reliant des nœuds sur un canevas, et les calculs sont confiés à pandas et scikit-learn. Partant de la version 0.5.8, les travaux décrits ici visaient trois objectifs : faire fonctionner l'application sur un Mac à puce Apple, faciliter l'arrivée d'un projet existant (données, bases de données, scripts Python) dans l'application, et réduire les temps de calcul sur les gros jeux de données.

Les principaux apports sont :

- deux nouveaux nœuds de lecture : **Lecteur Parquet / Excel** et **Lecteur SQL** (SQLite, PostgreSQL, MySQL) ;
- l'**import d'un script Python**, converti en schéma sans être exécuté, y compris sur plusieurs fichiers et avec le code de préparation des données ;
- une fenêtre **Données et scripts** avec une bibliothèque de scripts et l'import d'un dossier de projet complet ;
- des calculs répartis sur **tous les cœurs** du processeur et la suppression d'un double entraînement ;
- un comportement plus sûr des encodeurs sur les colonnes texte.

Le tout est traduit en anglais, documenté (français et anglais) et couvert par 15 nouveaux tests. L'ensemble représente environ 1 700 lignes dans de nouveaux fichiers et 480 lignes ajoutées aux fichiers existants.

1 Contexte et objectifs

KataML a été développé pour l'enseignement du machine learning. Chaque nœud du canevas correspond à une étape d'une analyse (lecture des données, préparation, modèle, évaluation) et le workflow complet s'exporte en script Python ou en notebook Jupyter. L'application est un serveur web local écrit en Python : le navigateur affiche l'interface, un processus séparé exécute les calculs.

Ce travail a été mené dans le cadre du **master Statistique pour l'évaluation et la prévision (SEP)** de l'Université de Reims Champagne-Ardenne, où KataML est utilisé comme support d'enseignement du machine learning. Son point de départ était une difficulté pratique : l'application ne semblait pas pouvoir être lancée sur un Mac. Une fois ce point éclairci, le travail s'est poursuivi sur un cas concret, le projet de **prédiction des retards de vols** réalisé dans le master (données de vols et de météo dans PostgreSQL, régression logistique avec scikit-learn), avec la question suivante : comment importer facilement un tel projet dans KataML pour le visualiser et le faire évoluer ?

Les objectifs retenus ont été :

1. lancer KataML sur macOS (puce Apple) ;
2. lire les formats de données courants (Parquet, Excel) et les bases SQL ;
3. reconstruire automatiquement le schéma d'un script Python existant ;
4. rassembler les données et les scripts d'un projet dans l'application ;
5. accélérer les calculs sur de gros volumes ;
6. corriger les erreurs rencontrées avec les colonnes texte.

2 Fonctionnement sur macOS

L'analyse du code a montré qu'**aucune modification n'était nécessaire** pour macOS. KataML est écrit en Python pur ; il ne dépend que de pandas et de scikit-learn, qui existent pour les Mac à puce Apple. Le processus de calcul utilise déjà le mode de démarrage `spawn`, qui est celui de macOS, et le script de lancement `kataml.sh` est compatible.

La difficulté venait de la distribution : les versions « portables » publiées (qui embarquent Python) n'existent que pour Linux (AppImage) et Windows (archive zip). Sur Mac, il faut passer par l'installation avec `uv`, déjà documentée :

```
curl -LsSf https://astral.sh/uv/install.sh | sh    # une seule fois
cd ~/Documents/master/kataml-v0.5.8
./kataml.sh                                        # le navigateur s'ouvre
```

Une version portable pour macOS (application `.app`) reste une piste possible pour le projet (voir section 11).

3 Chargement des données

3.1 Lecteur Parquet / Excel

Le nœud **Lecteur Parquet / Excel** (`io.table_reader`) lit les formats suivants ; le format est déduit de l'extension du fichier.

Format	Extensions	Fonction pandas
Parquet	`.parquet`, `.pq`	`read_parquet` (pyarrow)
Excel	`.xlsx`, `.xlsm`, `.xls`	`read_excel` (openpyxl)
Feather	`.feather`	`read_feather`

Pour Excel, un paramètre permet de choisir la feuille, par son nom ou son numéro (première feuille par défaut). Si une bibliothèque manque, le nœud indique laquelle installer. Les paquets `pyarrow` et `openpyxl` ont été ajoutés aux dépendances (`pyproject.toml` et `requirements.txt`). Comme tous les nœuds, ce lecteur s'exporte en code Python : `pd.read_parquet('vols.parquet')`.

3.2 Lecteur SQL

Le nœud **Lecteur SQL** (`io.sql_reader`) exécute une requête et renvoie le résultat sous forme de table. La base peut être :

- un **fichier SQLite** (`.db`, `.sqlite`) du dossier de données, ouvert en **lecture seule** : une requête ne peut pas modifier la base ;
- une **URL de connexion** SQLAlchemy vers PostgreSQL ou MySQL, par exemple `postgresql+psycopg://utilisateur@localhost:5432/db_vols`.

Le champ « requête » accepte une requête SQL, un simple nom de table, ou rien du tout s'il n'y a qu'une table ; en cas d'ambiguïté, la liste des tables disponibles est affichée. Un nombre maximal de lignes permet de travailler sur un extrait.

Sécurité. Écrire une URL contenant un mot de passe dans le nœud l'enregistrerait en clair dans le fichier du workflow. Le nœud accepte donc la syntaxe `$DATABASE_URL` : l'URL est lue dans la variable d'environnement du même nom, définie avant de lancer l'application.

```
uv pip install sqlalchemy "psycopg[binary]"        # pilote PostgreSQL, une fois
export DATABASE_URL=postgresql+psycopg://utilisateur:motdepasse@localhost:5432/db_vols
./kataml.sh
```

3.3 Import des fichiers

Deux améliorations complètent ces lecteurs :

- **dépôt sur le canevas** : un fichier déposé crée automatiquement le lecteur adapté (CSV, Parquet / Excel ou SQL selon l'extension) ;
- **bouton « Importer » de l'inspecteur** : il ne proposait que les fichiers CSV, y compris dans les nouveaux lecteurs, si bien que les fichiers Parquet ou Excel apparaissaient grisés. Il propose désormais les extensions du lecteur concerné.

4 Import d'un script Python en schéma

4.1 Principe

Cette fonctionnalité reconstruit le workflow équivalent à un script pandas / scikit-learn. Le script n'est **jamais exécuté** : il est analysé avec le module `ast` de Python, qui le transforme en arbre syntaxique. Un interpréteur « symbolique » parcourt ensuite cet arbre instruction par instruction et suit des *valeurs abstraites* plutôt que des données réelles. Cette approche a trois avantages : elle est sûre (aucun code n'est lancé), rapide, et fonctionne sans les données ni la base de données du projet.

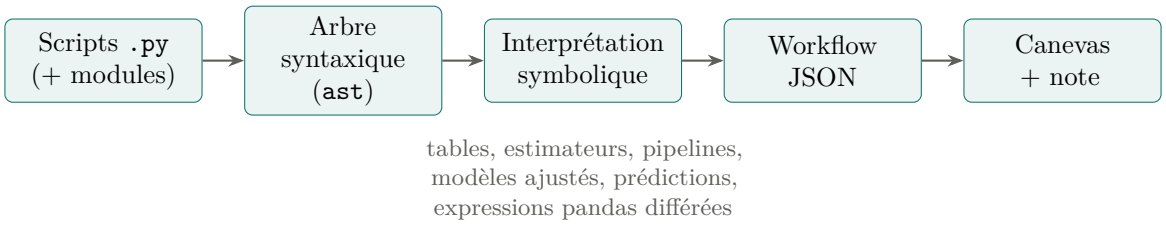


Fig. 1 : Chaîne de traitement de l'import de script (module `kataml/py_import.py`).

4.2 Valeurs suivies

L'interpréteur associe à chaque variable du script une valeur abstraite :

Valeur	Signification
`Tab`	une table produite par un nœud (sortie d'un port), avec son rôle : table, variables explicatives `X`, cible `y` ou prédictions
`Est`	un estimateur scikit-learn non ajusté et ses hyperparamètres
`Pipe`, `ColT`	un `Pipeline` (avec le nom de ses étapes) et un `ColumnTransformer` (transformeurs et colonnes)
`Fitted`	un modèle ajusté, c'est-à-dire un nœud dont la sortie est un modèle
`Deferred`	une expression pandas mise de côté (un masque, une série de dates), recopiée là où elle est utilisée
littéraux	constantes du script (nom de la cible, listes de colonnes, chemins)

4.3 Éléments reconnus

Code Python	Nœuds créés
`pd.read_csv`, `read_parquet`, `read_excel`	lecteur CSV ou Parquet / Excel
`pd.read_sql`, `create_engine`, `os.getenv`	lecteur SQL (requête reconstruite, URL en `$VARIABLE`)
`load_iris()`…	jeu de données scikit-learn
`dropna`, `drop(columns=…)`, `df[[…]]`	lignes incomplètes, filtre de colonnes
`train_test_split`	partitionnement (proportion, stratification, graine)
estimateurs, `Pipeline`, `ColumnTransformer`	transformeurs, modèle et nœud Pipeline
`fit`, `predict`, `predict_proba`, `score`	prédicteur
métriques `sklearn.metrics`	évaluation (classification, régression) ou courbe ROC
`cross_val_score`, `cross_validate`	validation croisée (stratégie et métriques)
`cross_val_predict`	prédictions hors pli
autres opérations pandas sur une table	nœud **Script Python** reproduisant le code

4.4 Points notables

Suivi des fonctions. Les fonctions définies dans le script sont suivies (« inlining ») : leurs paramètres reçoivent les valeurs abstraites des arguments et la valeur renvoyée est récupérée. Les pipelines ajustés étape par étape, à travers `named_steps["preprocesseur"]` puis `fit_transform`, sont réassemblés.

Plusieurs fichiers. Un projet réparti en modules (`from app.config.constants import ...`) est pris en charge : les fichiers importés par le script principal servent de bibliothèques, leurs constantes et fonctions sont reprises. Les modèles identiques construits deux fois (deux appels à `creer_modele()`) ne produisent qu'une seule série de nœuds.

Code de préparation. Les opérations pandas non reconnues deviennent des nœuds **Script Python** lisibles. Les variables intermédiaires (masques, dates) sont réinjectées dans le code, et les opérations sur deux tables utilisent les deux entrées du nœud (`df` et `df2`). Seules les chaînes pandas partant d'une table et produisant une table sont converties ; le code de calcul (numpy, métriques maison) est laissé de côté.

Liaison entre scripts. Lorsqu'un script écrit un fichier (`to_parquet`) qu'un autre relit, les deux parties sont reliées directement : la préparation et le modèle forment un seul schéma.

Transparence. Ce qui n'a pas pu être déduit (un chemin calculé, un paramètre non littéral) est listé dans une note jaune posée sur le canevas.

4.5 Exemple : le projet de retards de vols

Les deux scripts principaux du projet (`preparation.py` et `regression_logistique.py`) et leurs modules de configuration donnent un schéma de 22 nœuds reliés, sans aucune retouche :

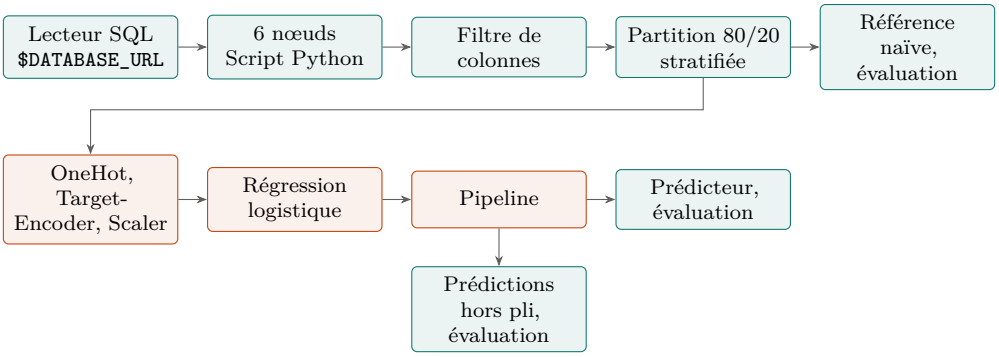


FIG. 2 : Schéma reconstruit à partir du projet de retards de vols (vue simplifiée).

Le lecteur SQL reprend la requête construite par f-string, la colonne cible `RETARD_15` et les listes de colonnes sont retrouvées dans le module de préparation, et le `TargetEncoder` des aéroports est placé dans le Pipeline comme dans le script d'origine.

4.6 Limites

L'analyse est statique : ce qui ne se décide qu'à l'exécution (un chemin construit dynamiquement, une boucle sur des modèles, une condition) est approché ou signalé dans la note. Les boucles ne sont parcourues qu'une fois et seule la première branche d'un `if` est suivie. Le nœud Pipeline de KataML accepte trois prétraitements au plus ; au-delà, le Pipeline n'est pas créé et la note l'indique.

5 Espace « Données et scripts »

Un bouton **Données** ajouté à la barre d'outils ouvre une fenêtre en deux parties (fichier `static/js/project.js`).

- **Données** (`workspace/data`) : fichiers CSV, Parquet, Excel et bases SQLite, avec leur type et leur taille. **Au canevas** crée le lecteur adapté ; on peut importer des fichiers ou les déposer dans la fenêtre, et retirer un fichier.
- **Scripts** (`workspace/scripts`) : une bibliothèque de scripts Python. On peut ajouter des fichiers, créer un script à partir d'un modèle commenté, l'éditer, puis cocher les scripts à convertir et cliquer sur **Créer le schéma**. Les autres scripts de la bibliothèque servent de modules s'ils sont importés.

Import d'un dossier de projet. Le bouton **Importer un dossier de projet** prend tout un dossier. Les scripts rejoignent la bibliothèque ; les dossiers `.venv`, `.git`, `__pycache__` et les caches sont ignorés, ainsi que les fichiers vides et les tests. Les fichiers de données sont seulement repérés : lors de la conversion, seuls ceux que le schéma lit réellement sont importés, ce qui évite d'envoyer

des centaines de mégaoctets inutiles. S'il manque des fichiers, KataML les liste et propose de les importer.

Routes ajoutées au serveur.

Route	Rôle
`/api/import_script`	conversion de scripts ; renvoie aussi les données manquantes
`/api/scripts`	liste de la bibliothèque de scripts
`/api/script/read`, `save`, `delete`	lecture, enregistrement, suppression d'un script
`/api/data/delete`	retrait d'un fichier de données

Ces routes réutilisent les protections existantes du serveur : jeton secret pour chaque requête et noms de fichiers validés, qui empêchent d'écrire en dehors du dossier de travail.

6 Performances

6.1 Calcul sur tous les cœurs

Les nœuds de validation croisée, de recherche d'hyperparamètres, de courbes et de permutations entraînent le modèle de nombreuses fois, mais KataML ne précisait jamais le paramètre `n_jobs` de scikit-learn : tout s'exécutait sur un seul cœur. Ces calculs sont désormais répartis sur tous les cœurs dans les nœuds suivants : validation croisée, prédictions hors pli, comparaison de modèles, recherche d'hyperparamètres (grille et aléatoire), courbes d'apprentissage et de validation, importance par permutation et test de permutation.

Le lancement de processus parallèles coûte environ une seconde. En dessous de **10 000 lignes**, le calcul reste donc sur un cœur, afin de ne pas ralentir les petits jeux de données des exemples de cours. Le nombre de cœurs se règle au lancement :

```
./kataml.sh --jobs 4          # 4 coeurs (-1 = tous, défaut ; 1 = un seul)
./kataml.sh --cache-mb 4096   # cache des résultats plus grand (1024 Mo par défaut)
```

6.2 Prédictions hors pli sans double entraînement

Le nœud **Prédictions hors pli** appelait deux fois `cross_val_predict` : une fois pour les classes prédites, une fois pour les probabilités, soit deux séries complètes d'entraînements. Pour les classifieurs dont la prédiction est exactement la classe de plus forte probabilité, la classe se déduit maintenant des probabilités :

$$\hat{y}(x) = \arg\max_k \ \hat{p}_k(x).$$

C'est le cas de la régression logistique, des arbres, des forêts, du boosting, des plus proches voisins, des bayésiens naïfs, du perceptron multicouche et des analyses discriminantes. Pour les autres (par exemple un SVC avec `probability=True`, dont les probabilités sont calibrées à part), les deux calculs sont conservés.

6.3 Mesures

Les mesures ci-dessous ont été faites dans un environnement de test à 2 cœurs, sur 60 000 lignes et 20 variables, avec une validation croisée à 5 plis. Les prédictions et les probabilités obtenues sont **strictement identiques** à celles de la version d'origine.

Modèle	Avant	Après	Résultats identiques
Forêt aléatoire (60 arbres)	138,7 s	44,0 s	oui
HistGradientBoosting	5,7 s	2,2 s	oui
Régression logistique	0,6 s	1,4 s	oui

Le gain croît avec la durée du calcul et le nombre de cœurs ; un Mac récent en compte au moins 8, ce qui laisse attendre un gain plus important. Pour un calcul très court comme la régression logistique ci-dessus, le coût de lancement des processus domine. Ces mesures n'ont pas encore été refaites sur le Mac lui-même.

7 Encodeurs et colonnes texte

Avec les données de vols, le `TargetEncoder` semblait refuser les colonnes texte (codes d'aéroports). En réalité, scikit-learn les accepte ; trois situations produisaient des erreurs :

1. **Cible non renseignée.** Le `TargetEncoder` calcule pour chaque catégorie la moyenne de la cible ; sans cible, scikit-learn renvoyait un message obscur (`Expected array-like ... got None`). KataML vérifie maintenant si un transformeur exige la cible (étiquettes de scikit-learn, complétées pour les sélecteurs de variables) et l'indique clairement.
2. **Colonnes laissées vides.** Un transformeur sans colonnes choisies s'appliquait à toutes les colonnes : une normalisation échouait alors sur du texte. Désormais, les encodeurs (`OneHotEncoder`, `OrdinalEncoder`, `TargetEncoder`) prennent par défaut les seules colonnes texte, et les normalisations et réductions de dimension (`StandardScaler`, `PCA`…) les seules colonnes numériques. Les colonnes retenues sont affichées dans les messages du nœud, et le code exporté reproduit ce choix.
3. **Colonne texte non encodée.** Une colonne texte qui arrive jusqu'au modèle provoque une erreur ; le message existant de KataML l'explique déjà.

Pour le projet de vols, le montage recommandé est :

Lecteur Parquet → Partitionnement → `TargetEncoder` (`ORIGIN`, `DEST`) → `OneHotEncoder` (`OP_UNIQUE_CARRIER`) → `StandardScaler` → régression logistique,

avec la cible `RETARD_15` renseignée dans chaque transformeur et les quatre nœuds réunis dans un Pipeline pour les prédictions sur le jeu de test.

8 Traduction et documentation

KataML est bilingue : l'interface et les messages sont écrits en français et traduits en anglais à l'affichage. Tous les nouveaux nœuds, paramètres, boutons et messages ont été ajoutés aux dictionnaires de traduction (`kataml/i18n.py` pour le serveur, `static/js/i18n.js` pour l'interface), y compris les messages composés (motifs d'expressions régulières).

La documentation MkDocs a été complétée dans les deux langues : barre d'outils (import de script, fenêtre **Données**, bases de données), dépôt de fichiers sur le canevas, options de lancement et historique des versions.

9 Tests

Trois fichiers de tests ont été ajoutés ; ils s'exécutent avec la suite existante (`uv run python -m unittest discover -v tests`).

Fichier	Tests	Contenu
`test_py_import.py`	9	conversion d'un script en workflow exécutable ; modules et constantes ; Parquet ; repli vers Script Python ; scripts invalides ; lecteur Parquet / Excel et code exporté ; lecteur SQL (requête, table, erreurs, variable d'environnement) ; script avec base de données ; bibliothèque de scripts
`test_performance.py`	3	réglage du nombre de cœurs ; classifieurs compatibles ; prédictions hors pli identiques à scikit-learn
`test_text_columns.py`	3	TargetEncoder sans cible ; choix automatique des colonnes et export Python exécuté ; absence de colonne texte

État de la vérification. La partie analyse de l'import de scripts a été vérifiée sur les cas des tests et sur le projet de retards de vols. La lecture SQLite et l'équivalence des prédictions hors pli ont été vérifiées dans un environnement de test. La syntaxe de tous les fichiers Python et JavaScript a été contrôlée. L'exécution de la suite complète et l'essai de l'interface restent à faire sur la machine cible, où les dépendances seront installées par `uv`.

10 Fichiers modifiés

Fichier	Lignes	Modification
Nouveaux fichiers		
`kataml/py_import.py`	1 135	import de scripts Python
`kataml/static/js/project.js`	271	fenêtre « Données et scripts »
`tests/test_py_import.py`	199	tests de l'import et des lecteurs
`tests/test_text_columns.py`	80	tests des encodeurs
`tests/test_performance.py`	51	tests des performances
Fichiers existants (lignes ajoutées / retirées)		
`kataml/nodes.py`	+326 / −11	lecteurs Parquet / Excel et SQL, cœurs, encodeurs
`kataml/server.py`	+66 / 0	routes d'import et de bibliothèque
`kataml/static/js/main.js`	+35 / −9	bouton Données, dépôt, import
`kataml/__main__.py`	+23 / −1	options de lancement (cœurs, cache)
`kataml/i18n.py`	+10 / −1	traductions (nœuds, messages)
`kataml/static/js/api.js`	+6 / 0	appels aux nouvelles routes
`kataml/static/js/inspector.js`	+6 / −3	bouton Importer selon le lecteur
`kataml/nodes_compose.py`	+3 / −2	calcul sur plusieurs cœurs
`kataml/nodes_explain.py`	+2 / −2	calcul sur plusieurs cœurs
`pyproject.toml`, `requirements.txt`	+4 / 0	dépendances pyarrow et openpyxl
`kataml/static/js/i18n.js`	—	traductions de l'interface
`kataml/static/css/app.css`	—	styles de la fenêtre « Données et scripts »
`kataml/static/index.html`	—	bouton « Données » de la barre d'outils
`docs/fr`, `docs/en`	—	barre d'outils, canevas, installation, historique

11 Perspectives

- **Visualisation des réseaux de neurones.** KataML propose déjà `MLPClassifier` et `MLPRegressor`, mais pas de vue du réseau. Un nœud dédié afficherait l'architecture (couches, neurones, poids des connexions) et la courbe de perte au fil des itérations.
- **PyTorch et TensorFlow.** Grâce aux bibliothèques `skorch` et `SciKeras`, qui donnent aux réseaux l'interface de scikit-learn, ces réseaux pourraient être intégrés comme une extension optionnelle, avec calcul sur le processeur graphique des Mac (MPS).
- **Version portable pour macOS**, sur le modèle des versions Linux et Windows.
- **Intégration au projet** : ces modifications peuvent être proposées à l'auteur de KataML par une demande de fusion sur GitLab, après exécution de la suite de tests.

A Commandes utiles

```
cd ~/Documents/master/kataml-v0.5.8
uv run python -m unittest discover -v tests     # suite de tests complète
./kataml.sh                                      # lancement (navigateur : Cmd+Maj+R)
./kataml.sh --jobs 4 --cache-mb 4096             # réglages de performance
uv run python -m kataml run workflow.json --rapport rapport.html   # sans interface
```